

IoT-Based Intrusion Detection System Using Machine Learning

Corrected technical status edition - 22 August 2026

Submitted by Mahi Patel (612303107), Malhar Falke (612303108), Yugandhar Pise (612303140), and Vaibhav Tayade (612303184), under the guidance of Prof. S K Gaikwad, Department of Computer Engineering, COEP Technological University.

Evidence status: this edition replaces the April 2026 report as the current technical account. Exact cross-dataset SFAF results are withdrawn pending a protocol-correct rerun. Live-model accuracy is validated only on synthetic traffic. Raspberry Pi performance remains unmeasured.

Executive summary

The repository now contains two explicitly separate prototypes. The live edge IDS/IPS converts packets into a versioned 22-feature flow representation and classifies ten traffic classes with an ONNX model. It supports offline pcap, timed replay, and live interface capture, then aggregates flow verdicts into incidents for alerting, dashboard display, throttling, or blocking. A second research pipeline aligns heterogeneous tabular datasets into a 12-feature binary space to study cross-dataset transfer.

The April 2026 report treated these prototypes as one validated deployment story and reported exact SFAF performance from an evaluation whose diagonal cells trained and evaluated on the same rows. That resubstitution invalidates the in-domain diagonal and contaminates comparisons derived from it. Those claims and artifacts are preserved under `legacy/resubstitution-results/` but are no longer presented as current evidence.

The engineering system is substantially stronger after review: packet direction follows the observed initiator, pcap timing and link types are validated, fragments are handled safely, model contracts are machine-checked, incident state is bounded, IPS timeouts refresh correctly, dashboard access is authenticated, dataset archives fail closed, and development plus Pi dependencies are locked. The main unresolved scientific task is to mount the public datasets and rerun the corrected protocol. The main deployment task is to collect real labelled traffic and benchmark on an actual Raspberry Pi.

1. Scope and evidence policy

This document distinguishes implemented capability from scientific evidence. An implementation can be correct without proving real-world detection quality; a fast host benchmark can be useful without proving target-hardware capacity. Every result is therefore labelled by evidence scope.

Evidence class	What is currently supported	What it does not prove
Hermetic tests	58 tests cover parsing, flow state, model loading, IPS, dashboard, data handling, and regressions	Accuracy on real networks

Evidence class	What is currently supported	What it does not prove
Synthetic held-out scenarios	70 scenarios held out from training; ONNX and C parity checked	Generalisation beyond the traffic generators
Host benchmarks	ONNX, C, extraction, end-to-end, and memory measurements on Apple Silicon	Raspberry Pi throughput or soak stability
Cross-dataset study	Corrected code and evaluation protocol	Current exact metrics until datasets are remounted and rerun

The previous exact in-domain AUC, generalisation-gap magnitude, transform ranking, and small-label calibration claims are withdrawn. The historical off-diagonal result remains qualitative evidence that domain shift is severe, but it is not the current full-protocol result.

2. System architecture

The live path is deliberately small and shares one feature extractor between training and serving:

```
pcap / timed replay / live interface
    -> packet parser
    -> locked flow table
    -> 22-feature contract v2
    -> ONNX detector
    -> per-source incidents
    -> alert log + dashboard + optional IPS response
```

`src/flow_features.py` is the single packet-to-feature implementation. The trainer, corpus builder, daemon, tests, and C exporter use the same feature names and semantics. `src/ids_daemon.py` validates model purpose, runtime compatibility, exact feature order, feature count, and semantic contract before creating an ONNX session. `src/ips_response.py` implements the response ladder; `src/dashboard.py` provides the read-only operator view.

The research path is separate:

```
public flow datasets
    -> dataset-specific loader and quality report
    -> 12-feature SFAF alignment
    -> deterministic dataset splits
    -> cross-dataset and leave-one-dataset-out evaluation
```

The 12-feature binary research artifact is not accepted by the live daemon. This boundary prevents a scientifically withdrawn model with incompatible inputs from being used for enforcement.

3. Packet and flow feature contract

The live model consumes 22 raw flow features: protocol, duration, packet and byte totals, packet and byte rates, packet-length statistics, inter-arrival statistics, TCP-flag ratios, directional ratios, host-context counts, and destination port. Feature contract version 2 records corrected direction semantics.

Flow identity uses a canonical bidirectional endpoint key, but forward and backward direction do not. Direction follows the first observed sender unless the first packet is SYN+ACK, which proves that sender is the responder and allows initiator inference. A capture beginning midstream without a handshake cannot prove the original initiator, so first observed sender remains an explicit fallback.

The parser supports Ethernet, VLAN, QinQ, IPv4, IPv6, TCP, UDP, and ICMP. Classic pcap microsecond and nanosecond timestamp formats are distinguished. Unsupported link types are rejected instead of being decoded as Ethernet. Non-initial IP fragments are ignored because their payload does not carry a reliable transport header; IPv6 extension-header lengths are validated.

4. Live model and synthetic validation

The deployable model is `models/live_ids.onnx`, with metadata in `models/live_meta.json`. It is a ten-class XGBoost model trained from synthetic scenarios generated by the repository. The classes are benign, portscan, synflood, icmpflood, udpflood, SSH brute force, slowloris, Mirai, Xmas scan, and MQTT flood.

Model fact	Current value
Purpose	live_multiclass_ids
Runtime compatible	true
Feature contract	22 features, semantic version 2
Training split	280 scenarios train / 70 scenarios test
Training flows	44,761 capped flows
Held-out flows	12,191 flows
ONNX size	91.8 KB in the latest benchmark
ONNX SHA-256	0a62b0e5c7ad98c8172a0032bd0e6b55e6fb2208defeee2cc773b21b1f2d1336

The scenario-held-out test reports macro-F1 1.0000. An independent unseen-seed benchmark reports 99.65% multiclass accuracy and 100% attack detection, with Mirai recall 94.2%. Removing destination port leaves the scenario-held-out score unchanged. These results demonstrate consistency and separability of the synthetic generator, not real-network accuracy.

ONNX output is compared against the XGBoost source model before shipment. The C header is separately compared against the booster. The model is deleted or rejected when parity gates fail.

5. Detection, alerting, and prevention

Flow verdicts are aggregated by source and attack type so the operator receives incidents rather than a line for every flow. Incident watermarks retain only keys active in the current window, bounding memory and allowing a genuinely new recurrence to alert after an earlier incident disappears.

The optional IPS response ladder supports monitor, throttle, and block stages. Network scope is explicit: host scope protects the Pi itself; network scope is appropriate only when the Pi is inline and can affect forwarded traffic. Repeated detections refresh in-memory expiry, persisted state, and firewall timeout. nftables throttling uses per-source meters rather than one shared bucket. IPv4 and IPv6 rules are generated separately.

Prevention remains opt-in. Dry-run is the default demonstration mode, and allowlists plus confidence thresholds reduce self-inflicted blocking risk.

6. Dashboard and operational security

The dashboard binds to loopback by default. Network exposure requires either a token or an explicit insecure override. The authenticated page reads the token from the initial query, stores it only in session storage, removes it from the visible browser URL, and sends it in the X-Auth-Token header for API polling. Setup creates a mode-600 token file and passes its path to the service rather than exposing the token in process arguments or systemd unit metadata.

Alert logs rotate, dashboard reads are incremental, and persisted IPS state is reloaded safely. The Pi installer has an executable strict preflight path that checks initialization without installing packages or writing services.

7. Corrected SFAF research protocol

The cross-dataset evaluator now creates one deterministic stratified 80/20 split per dataset. A model is fitted only on the 80% training partition. Its diagonal cell is evaluated on the untouched 20%; off-diagonal cells use an independent target dataset. Output records evaluation type plus train and evaluation sizes.

Alignment no longer silently discards a row because one supplied feature did not parse numerically. Missing values remain visible and an alignment report records input rows, output rows, dropped rows, row-level missingness, and per-feature missingness. Bot-IoT sampling uses a memory-bounded random-priority reservoir across all chunks instead of taking an ordered prefix. IoT-23 caches include a digest of the loader implementation, so parsing or taxonomy changes select a new cache.

Small-budget threshold transfer includes every requested repeat. A single-class calibration draw uses the deployed 0.5 threshold rather than being silently omitted; reports show total repeats, calibrated draws, and success rate.

The dataset symlink currently points to an unavailable external volume. No replacement metric is fabricated. The corrected experiment commands are ready for execution after the data mount returns.

8. Dataset and dependency supply chain

Dataset downloads use normal TLS certificate verification. Kaggle no longer performs blind extraction. Tar and ZIP members are resolved against the target directory before extraction; absolute paths, traversal, links, and special files are rejected. Direct archives print SHA-256 and are not extracted without a trusted digest or explicit `--allow-unverified` risk acceptance.

Research, development, and Pi environments have separate reviewed inputs and exact transitive Python 3.10 locks. The audit added undeclared `pyarrow` and `psutil` dependencies, removed unused direct declarations, verified the full development lock in a fresh virtual environment, and confirmed CPython 3.10 aarch64 wheels for NumPy and ONNX Runtime. The locks pin versions but do not yet pin wheel hashes.

9. Performance status

The latest host benchmark measured 11.1 microseconds mean single-flow ONNX latency, 399,270 flows/s at batch 1024, 1.111 microseconds per flow for the native C model, 208,465 packets/s for parse plus aggregation, 51,321 flows/s end-to-end on the demo pcap, and 56.6 MB daemon RSS. These are Apple Silicon measurements and are not Raspberry Pi results.

The benchmark still prints a clearly marked `host-times-12` Pi projection for planning. It is not an acceptance result. The target gate is to run the same benchmark on a 64-bit Pi, preserve raw output, exercise realistic packet sizes and rates, and complete a soak run while watching dropped packets, memory, temperature, and incident latency.

10. Verification and reproducibility

The hermetic suite currently contains 58 tests. It covers model loading, feature parity, packet formats, fragments, flow lifecycle, thread safety, bounded caches, dashboard authentication, log rotation, IPS rule generation, state refresh, safe dataset extraction, corrected research splits, cache versioning, threshold transfer, model exports, deployment script syntax, and the end-to-end demo.

CI installs `requirements-dev.txt`, regenerates all dependency locks to detect input drift, runs Ruff defect checks, executes the suite without an external dataset mount, validates ONNX metadata and dimensions, compiles the C model, runs the end-to-end demo, and rejects benchmark reports with failed sections.

Reproduction order after mounting datasets:

```
python code/download_datasets.py --check-only
python code/multidataset.py
python code/cross_dataset_eval.py
python code/transfer_experiment.py
python code/threshold_transfer.py
python code/02_train_sfaf.py
```

Raw inputs, package lock, git commit, split seeds, sample sizes, class balance, alignment-quality reports, metrics, plots, model metadata, and checksums should be archived together for the next academic result set.

11. Limitations and unresolved risks

- Real labelled packet captures have not yet passed through the complete

packet-to-feature-to-verdict pipeline.

- The SFAF study has corrected code but no current protocol-correct metrics

because the external dataset volume was unavailable.

- Perfect or near-perfect synthetic performance is not evidence of deployment

accuracy, concept-drift resilience, or adversarial robustness.

- Raspberry Pi throughput, packet loss, thermal behavior, and long-run memory

stability are unmeasured.

- Midstream flow direction is necessarily heuristic without a handshake.
- Unsupported capture link types fail safely but are not yet decoded.
- Python lock versions are exact, but wheel hashes remain unpinned.
- Public-repository governance, license, data cards, model cards, branch

protection, and automated security settings require completion.

- Historical large binary objects remain in Git history; removing them would

require a coordinated destructive rewrite.

12. Prioritized future development

Priority	Development option	Acceptance evidence
P0	Remount datasets and rerun corrected NxN, LODO, and threshold-transfer studies	Raw result bundle, no train/eval overlap, quality report, reproducible commit
P0	Capture real labelled traffic through the live 22-feature extractor	Grouped device/scenario/time split and per-class error analysis
P1	Benchmark and soak on Raspberry Pi	Raw benchmark, packet-drop counters, temperature, RSS, incident latency
P1	Decide how the research and live feature spaces should converge	Written ADR plus validated adapter or continued separation
P1	Add license, model card, data cards, security policy, and protected-branch controls	Repository files and verified GitHub settings
P2	Add supported Linux cooked and radiotap capture decoders	Format fixtures and parser parity tests
P2	Evaluate drift, adversarial traffic, calibration, and abstention	Pre-registered protocol and independent holdout

Conclusion

The project is now an honest engineering prototype rather than a collection of unqualified accuracy claims. The live IDS/IPS has a coherent, versioned feature contract, verified model exports, safer operations, and strong hermetic test coverage. The research pipeline has a corrected evaluation design and explicit data-quality accounting. Neither synthetic accuracy nor host speed is presented as proof of real-world or Raspberry Pi performance.

The next defensible milestone is evidence, not another model tweak: rerun the public-dataset study without resubstitution, process real labelled packet captures end to end, and measure the target Pi directly. Those results can then support a final academic report without reviving the withdrawn claims.

Repository references

- README.md - project entry point and current evidence caveats.
- vault/Remediation_2026-08-22.md - issue-by-issue decisions and verification.
- vault/Reference/Architecture.md - two-prototype architecture.
- models/README.md and models/live_meta.json - artifact contracts.
- demo/results/BENCHMARK.md - host measurements and Pi acceptance gate.
- demo/results/CROSS_DATASET_FINDINGS.md - withdrawn-result notice and rerun state.
- deploy/README_PI.md - target deployment runbook.